Ricardo Miguel Nunes Camacho de Matos

Degree in Electrical and Computer Engineering Sciences

Protocol's Behaviour Analysis

Dissertation plan submitted in partial fulfillment
of the requirements for the degree of

Master of Science in
Electrical and Computer Engineering

Adviser: Rodolfo, Professor Auxiliar,
NOVA University of Lisbon

FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE NOVA DE LISBOA

August, 2017

Abstract

Resumo

Contents

List of Figures ix

List of Tables xi

Listings xiii

1 Introduction 1
- 1.1 Motivation . . . 1
  - 1.1.1 Objectives and Contributions . . . 2
- 1.2 Report Structure . . . 3

2 Related Work 5
- 2.1 Introduction . . . 5
  - 2.1.1 Types of Data . . . 6
  - 2.1.2 Types of Mining . . . 7
- 2.2 Introduction to Sequential Pattern Mining . . . 9
  - 2.2.1 Problem definition . . . 9
  - 2.2.2 Taxonomy . . . 11
  - 2.2.3 Structure definition . . . 14
  - 2.2.4 Architectures . . . 15
- 2.3 Sequential Pattern Mining Algorithms . . . 15
  - 2.3.1 GSP . . . 16
  - 2.3.2 SPADE . . . 18
  - 2.3.3 PrefixSpan . . . 20
- 2.4 Conclusions . . . 22

3 Work Plan 25
- 3.1 Task Description . . . 25
- 3.2 Developed Work . . . 26
- 3.3 Schedule . . . 26

4 Conclusions 29

Bibliography 31

List of Figures

2.1 Sequential pattern mining Architecture 16

3.1 Schedule. 26

List of Tables

2.1 Sequential database. 10
2.2 Candidate Generation Example. 18
2.3 Vertical database. 19
2.4 Vertical to Horizontal database. 19
2.5 PrefixSpan database examples 20
2.6 S-Matrix 21

Listings

Chapter 1

Introduction

Over recent years the need to extract and organize information has become essential, and has been heavily demanded in today's market. With the increasing number of smart devices that can communicate with each other, the size of databases has increased exponentially and the amount of information to process and analyse has simply become too much to handle.
Keeping in mind the amount of time spent analysing information and the performance of the available resources, data-mining has gained an important role in data analysis, with a plethora of algorithms capable of running these tasks autonomously .
In telecommunications the problem of data gathering and analysis has also become relevant. The amount of data and signalling transmitted between different protocols in a given period of time can be hard to analyse and, more importantly, hard to identify patterns that could be relevant to better understand the current operational conditions. Consequently, there is a need to find a reliable approach to analyse the behaviours of different protocols and take conclusions about the obtained results from the data that is available.

1.1 Motivation

The analysis of the data generated in a telecommunication network is the motivation for this thesis, that takes into account three major cornerstones, being the first one providing a reliable Technical Diagnosis. With the rise in the volume of data, the time used in diagnosis has become too extensive, consuming many people and resources in order to fulfil a specific analysis. The goal here is to make diagnosis faster and more accurate, making problem resolution simpler. For instance in a specific network if packet drop rate

increases we want to know what caused the increase as quickly as possible in order to solve the problem before it escalates into something more serious.
Big Data can have an important role in selecting important data, since it can label sets of data into different groups, therefore making it easier to analyse in detail each of these individual sets, and can give additional information that otherwise could be inaccessible in naked eye.

Another important motivation is planning. Given that observing a series of raw data can be very complex and time consuming, predicting what will happen in the future to the system according with the known data may be even more complicated. This thesis aims to aid in the decision making of plans and procedures in order to identify in an early stage what is happening in the system and if that behaviour will later affect the performance or damage the system as a whole. This may be useful in multiple scenarios. Imagine several packet drops are observed, and by analysing the pattern of the messages you conclude that if the system keeps going on that direction it will eventually crash, within a week. With the aid of a data mining analysis you can take additional decisions to avoid the crash. This can be achieved by planning the system with parameters that will directly affect the future outcome, therefore preventing the crash.

Finally, by providing all of the above solutions, this thesis also intends to improve the security of a given system and prevent external intrusion to information that it may store in a database. This is accomplished again by analysing the behaviour of the system, and finding relations in the patterns that are extracted. If we get a frequent pattern that is deviating from the normal behaviour of the system, we can conclude that something is wrong and that an attack may be in course and it is needed to act on it. By making this verification we are ensuring the integrity and security of our system.

These three points are the major motivations for this thesis, but besides them we can also aim to hit the market with a solution to these problems, since protocol behavioural analysis in telecommunications is an area that has not been deeply researched. Additionally it is vital to maintain a system functioning with little to no errors and safe from attacks, and this can only be done by analysing permanently how the system is behaving.

1.1.1 Objectives and Contributions

This subsection describes the objectives that were defined for this thesis as well as a few contributions of this work.

Objectives:

- Elaborate a bibliographic survey on different algorithms that are able to categorize large amounts of data, and extract useful information and knowledge from them.

The goal here his study which algorithms can be used to classify a series of raw data, so the analysis becomes simpler, by extracting information in form of patterns that can predict the behaviour of the system.

- Propose a data categorization framework with low computational complexity, in order to process faster.
- Be efficient and accurate in categorizing the data. To do so it will require an extensive read on the selected algorithms for the pattern extraction to identify the most efficient ones and use only those who grantee better results.
- Apply classic algorithms to innovative scenarios in telecommunications. Here the algorithms chosen that meet the above requirements will be applied to analyse telecommunications Protocol Behaviours. To do so it will be necessary to use sequential pattern mining algorithms and try to incorporate their functionalities and use them in the data selected from message logs provided by these protocols.

Expected Contributions:

- Implement different sequential pattern mining algorithms in the context of computer networks. These algorithms will be used to find frequent patterns in telecommunications protocols, such as SIP and GSM signalling.
- Classify protocol behaviours as 'normal' behaviours or as 'deviations' from their normal state. This analysis in behaviour can have many different applications such as protocol's security, by analysing how a certain protocol is expected to behave, or anomaly detection.
- Identify in real time when a deviation of behaviour occurs, always with the aim to understand why it happened in the first place and where to act to correct it.

1.2 Report Structure

This thesis is organized as follows. Chapter 2 outlines the related research in sequential pattern mining, where the basic notions of data mining are introduced, as well as a description of the most used sequential pattern mining algorithms used in this thesis, alongside a performance analysis between them. Chapter 3 describes the research plan and the milestones to reach. Finally, Chapter 4 enumerates the conclusions of this report.

Chapter 2

Related Work

2.1 Introduction

Sequential pattern mining (SPM) was first introduced by Agrawal and Srikant [1]. In order to understand how this technique works lets start by showing a practical example. Assuming the 4 sequences: <(ab)>, <(abc)>, <(abc)d> and <(abc)dc>, the goal is to discover which of these sequences (or their subsets) are frequent. To accomplish this it is necessary to determine the frequency with which items belong to each sequence. A minimum frequency, refered to as minimum support, is usually considered. Looking again at the sequences, and assuming a minimum support of 3, we can classify the sequence <(abc)> as frequent since it appears three times, in <(abc)>, <(abc)d> and <(abc)dc>. The same can be said of <(ab)>, which is present in all four sequences. On the other hand, considering the sequence <(abc)d>, we observe that it reaches the minimum support threshold, since it does not verify the minimum support equal to 3, appearing only in the last two sequences.

In a more formal way, sequential pattern mining is defined by several other authors [12][19] as a data-mining technique that discovers sequences of frequent item-sets. These sequences can be seen as specific patterns in a sequence database, and the goal of sequential pattern mining is to extract those sequential patterns whose support exceeds a pre-defined minimum support. All the items in one item-set must have the same time stamp or happen within a certain pre-defined window of time in order to be associated to a sequence. A typical example of a sequential pattern is "*Customers who buy a digital camera are likely to buy a color printer within a month*".

Sequential pattern mining has become a very popular data-mining technique with a

vast range of applications, such as customer purchase patterning analysis, correlation analysis, and software bug tracking or in the case we present in this thesis, protocol behaviour analysis. To better introduce SPM we first need to learn some core concepts of this data-mining technique.

2.1.1 Types of Data

There are many types of data that can be mined with mining techniques. Because different data is available, the knowledge extracted will also be different, which is very useful to take different conclusions about the problem we are trying to understand.
Essentially we can categorize data into two different types: **Qualitative**, and **Quantitative**. In qualitative type data information is characterized by a non-realistic value, because their attributes cannot be measured. An example of qualitative data are colors. If we want to describe a color we can define it as a code (e.g. Black #000000). By doing this we are transforming colors into *categorical variables*, where we assign a number that without a context has no meaning at all, but grouped in the category of numbers we can understand what they mean. However, in this type of data it does not make sense to determine a sum of colors or an average, since it has no meaning.

On the other hand, quantitative data can be quantified and verified, being used in statistical manipulation. An example of such data can be seen looking at the example of a shopping cart, where we can define the price of the items and apply arithmetic. In a short we can say that quantitative data defines, whereas qualitative data describes. With that in mind we can classify data-mining in different categories as defined in [19].

- **Relational database** – Here data is described by a set of attributes which are stored in tables, being a highly structured data repository. Data mining in these relational databases focuses mainly on discovering patterns and trends, being used in banks and in supermarkets to acquire customer behavioural information, such as customer spending habits, and profiling clients into categories such as high and low spenders.

- **Transactional database** – Refers to a collection of transactions (e.g. sale records), where data-mining enters to find a correlation between items and transaction records. These types of databases have gained popularity due to the increase in e-commerce, with a fast growing community of people that start to make all of their shopping online.

- **Spatial database** – Contains traditional data as well as geographic information of the corresponding data. It can be used to associate a certain geographic area with a specific behaviour.

- **Temporal and time-series database** – For each temporal data item, the corresponding time is associated. Can extract more useful information from an individual, by analysing it's temporal behaviour.

Looking at the problem that this thesis aims to solve, we can define our initial database as a *time-series*, since in telecommunications when a protocol is running the packets sent and received by a terminal are logged into several files across the network with all the important meta data associated to that instance such as ID, type of message, source, destination and finally a time-stamp.

It is evident that a time stamp is an important attribute to any dataset and, consequentially, important to data mining since it can add more knowledge to the process of extracting information. For that reason, sequential pattern mining has an important role in these types of databases, since it is used to find the relationships between occurrences of sequential events, although it is used in a special case of time series databases known as Sequential databases that can come whit or without a concrete notion of time.

2.1.2 Types of Mining

In the context of data mining it is possible to identify several types of mining, each of which applies different techniques to the data, extracting distinct knowledge. Data mining can be classified into two separate classes, defined in [19] as: descriptive mining that has the goal to summarize general properties of data, and prescriptive data that per forms inference on current data to make predictions of what will come next.
Given these two classes of data mining we can identify several techniques, reported with a few examples.

Association Rules – First introduced in [3], an association rule aims to extract interesting correlations, frequent patterns and associations between sets of items in a database. In a more formal explanation let $L = I_1, I_2, ..., I_n$ be a set of items. An association rule means a relationship of the form $X => I_j$, being X some set of items from L and I_j a single item not belonging to X. The rule is satisfied if for a confidence factor 0<c<1 at least c% of the transactions in a database satisfy both X and I_j.

Now looking at a more tangible example let's assume we have the following statement "70% of the people who buy running shoes also buy sports water bottle." Here we can observe that the antecedent of the rule is "running shoes", the consequent "sports water bottle" and finally the confidence factor is set at 70%. Given this information a sports outlet could rearrange it's stock and sales according to the customer habits. This type of mining technique is widely used in areas such as telecommunications.

Classification - Described in Han and Kamber (2000)[8], the objective of this technique is to build automatically a model that can classify a class of objects, in order to predict the classification of future objects. Data classification can be seen as a two-step process. In the first step a classifier is built to describe a set of predetermined data classes, being called the **learning step**. Here a learning algorithm is used to build the classifier by analysing a set of data called **training set**. In this collection of training data the classes and concepts that they belong to are predefined and provided so we can also define this step as **supervised learning**. The final step is used to test the classification accuracy by using the model to classify other test examples. Since the classifiers tend to overfit the data a **test set** is used to be classified by the model. The final accuracy will depend on the percentage of accurate classifications the model does to the test set.
An example of a classifier could be a model to categorize bank loan applications as *safe* or *risky*, or catalogue a car as *Sports car* or *Family van*.

Clustering - Contrary to the classification method, clustering uses an **unsupervised learning** process, which means that there are no predefined classes to the initial data set (training set) which is very common in large databases, because assigning classes to every single object can become quite costly. As defined by Han and Kamber (2000) [8] clustering is the process of grouping a set of physical or abstract objects (data) into classes of similar objects also known as **clusters**. Objects within a same cluster have high similarity, but are very dissimilar to objects in other clusters. **Similarities** are defined by functions, usually quantitatively specified as distances or other measures defined by domain experts.
Clustering applications are manly used in market segmentation, that categorize customers into different groups so that organizations can provide personalized services to each group.

Regression analysis - This method is the most widley used aproach for *numeric prediction*, which is the the task of predicting continuous (or ordered) values for a given input [8]. Regression involves learning a function that maps a data item to a real-valued prediction variable [7], and can be used to model the relationship between one or more *independent/predictor* variables and a *dependent/response* variable. In the context of data mining the predictor variables are the attributes of interest describing the tuple (i.e., making up the attribute vector). Generally the values of the predictor variables are known, being the response variables predicted by these methods.
Regression has a vast range of applications focusing on prediction. The fields of work can range from Buiseness such as predicting customer demands for a new product or in medicine by estimating the probability that a patient will survive given the results of a set of diagnostic tests [7]. Many regression-based methods exist such as P oisson regression, log-linear models and non-linear regression, although most of the problems can be solved simply by using linear regression.

Summarization - Consists in methods used to find a compact description for a subset of data[7]. This type of techniques are often used in interactive exploratory data analysis and automated report generation. Data summarization is very important in data preprocessing and is used for measuring the central tendency of data (e.g. mean, median and mode), measuring the dispersion of data (e.g. range, variance and standard deviation) and graphical representations such as histograms and scatter-plots are also useful to facilitate the visual inspection of the data [8].

Anomaly detection - This method builds models of normal network behaviour defined as *profiles* which are then used to detect the presence of new patterns that significantly deviate from the generated profiles [8]. These deviations may represent actual intrusions in the system or simply new behaviours that are not included in the profile. The advantage of this technique stands on the fact that it can detect novel intrusions that have not yet been observed, a human analyst needs to sort through all the deviations to ascertain which ones are real intrusions. A limitation of this method is the high percentage of false positives, which can be somewhat corrected by adding the new behaviours of the system to the profile. The new patterns of intrusion can be added to the set of signatures for misuse detection.

All these mining techniques are still used until this day and remain effective depending on the paradigm they are inserted into, having many research papers dedicated to study each of these techniques in detail. The next section of this chapter will bring more light to the case in study in this thesis, sequential pattern mining.

2.2 Introduction to Sequential Pattern Mining

2.2.1 Problem definition

This section defines the problem of mining sequential patterns, and also give some insights on the concepts used in this area. The basis for this definition was taken from [12], and can be viewed as follows. Given:

(i) A set of a set of sequential records (sequences) representing a sequential database D.

(ii) A minimum support threshold.

(iii) A set of k unique events or items $I = i_1, i_2, \ldots, i_k$.

The problem of sequential pattern mining consists on finding the set of all frequent sequences S, in the given database that supports the minimum threshold (minimum

support), in order to describe the data or predict future data. In order to better understand this problem we introduce different definitions that represent the roots of different techniques capable of mining sequential patterns.

Definition 1 *Let $I = I_1, I_2, I_3, ..., I_n$ be the set of all **items**. A **item-set** can be viewed as a non-empty, unordered collection of items.*

The item-set is denoted (x_l x_2 ... x_q), where x_k is an item. The brackets are omitted if an item-set has only one item, that is, (x) is written simply as x. An example of an item-set can be seen in a grocery store, where the items can be represented by a list of products bought at the same time such as bread, milk and cheese, represented by (abc). Note that the items all belong in the same event since they were all bought at the same transactional time.

Definition 2 *A **sequence** is a lexicographically ordered list of item-sets (events), such as $S = <e_1\ e_2\ e_3\ ...\ e_l>$ where the order of the item-sets has an important role, in this case $e_1$occurs before e_2, which occurs before e_3, and so on. The event e_j is also called an element of S [8].*

Let S be represented by <(ab)b(bc)c(cd)>. An item can occur only once in a given item-set, but it can have multiple instances in different item-sets of a sequence. In sequence S the item b has three instances, appearing in the item-sets (bc), (b) and in (ab). However if we were to add the item-set (dcd) to the sequence S it would not be possible, since it contains two instances of the same item.

Definition 3 *Given a sequence S its **support**, denoted as $\sigma(S)$, can be defined as "The total number of sequences of which S is a subsequence divided by the total number of sequences in the database D", whereas the **support count** of a sequence S is the total number of sequences in D of which S is a subsequence.*

A sequence can only have the classification of *frequent* if its support count (frequency) is greater or equal to the user-defined *minimum support*. An example of what a sequential database may look like is shown in table 2.1.

Table 2.1: Sequential database.

Sequence ID	Sequence of items
1	<a(abc)(ac)d(cf)>
2	<(ad)c(bc)(ae)>
3	<(ef)(ab)(df)cb>
4	<eg(af)cbc>

Looking at table 2.1 we can observe that the sequential database is composed of a set of tuples, <SID,S>, where SID is the Sequence ID, and S is the Sequence of items.

Definition 4 *A subsequence S' of S is denoted by* $S' \subseteq S$. *Formally, a sequence* $a=<a_1, a_2...a_n>$ *is a subsequence of* $b=<b_1 b_2...b_m>$, *and b is a supersequence of a, if there exist integers* $1 \leq i_1 < i_2 < \ldots < i_n \leq m$ *such that* $a_1 \subseteq b_{i1}$, $a_2 \subseteq b_{i2}$, $\ldots$, $a_n \subseteq b_{in}$ *[4].*

Finnally, we can have a formal definition of **Sequential pattern**, which is a sequence of itemsets that frequently occurred in a specific order. All items in the same itemsets are supposed to have the same transaction-time value or be included within a time gap [19], which will be defined as *max-gap*. If you watch a client transaction sheet all of its transactions can be viewed together as a sequence, where each individual transaction can be represented as an item-set of that sequence, that are ordered in a certain transaction-time.

2.2.2 Taxonomy

This thesis adopts the taxonomy defined in [12], where sequential pattern mining algorithms are categorized in three main categories: *Apriori-based*, *Pattern-growth* and *Early-prunning*. This taxonomy is used to classify sequential pattern mining algorithms based on the features that the different techniques provide. It also helps to better understand and distinguish these techniques before-hand, having some previous knowledge of the characteristics and capabilities of a given algorithm, by looking at what category it belongs.
This section gives examples in some of the key features from each of the categories mentioned above. The focus is on the features that are intrinsic to the algorithms, which include, SPADE[18], GSP[15] and PrefixSpan[10].

Apriori-based: Algorithms that fit into this category depend mainly in the apriori property which states that *"All nonempty subsets of a frequent itemset must also be frequent"* [12]. Besides that apriori algorithms are also described as being *antimonotonic* or *downward-closed*, which means that if a given sequence cannot pass a pre-defined minimum support test, all of its supersequences will also fail the test, and consequently will be discarded as candidates of frequent sequences. There are three key features that define apriori-based methods which are discussed bellow:

1. **Breadth-first search:** This feature is used to describe apriori-based algorithms because all of the k-sequences are constructed together in each kth iteration of the algorithm as it travels trough the data. So suppose you want to construct the length-3 sequences. In this method all of the length-2 sequences must be solved first in order to proceed. This makes available a lot of information that can be used in pruning, but comes with a trade-off between information availability and memory consumption.

2. **Generate-and-test:** Introduced by the Apriori algorithm in [3], uses exhaustive join operators, where the pattern continues growing one item at a time, being always tested against the minimum support to ensure the given sequence is frequent. This causes the generation of an enormous number of candidate sequences, that can consume a lot of memory in the early stages.

3. **Multiple scans of the database:** As the name suggests this feature is based on scanning the original database every time it needs to ascertain if the generated candidate sequences are frequent. This scanning keeps on going as long as there exists candidate sequences that need to be verified. This method is very costly in terms of processing time and I/O. A solution to this limitation comes with the introduction of the next category.

Pattern-Growth: Algorithms that fit this category usually display better performance than apriori-based, both in processing time and in memory consumption. This method was implemented to serve as a solution to the problem of *generate-and-test* that was present in the early apriori algorithms. The goal of this technique is to avoid the candidate generation step altogether, and the search is only done in a restricted portion of the initial database. In order to fulfil the needs of pattern-growth algorithms different key features must be considered, including:

1. **Candidate sequence pruning:** Pattern-growth algorithms like PrefixSpan and others, attempt to use a data structure that allows pruning of candidate sequences in the early stages of mining. Algorithms that do so have a smaller search space, which in return results in a more directed and narrower search procedure. This feature is also present in early-pruning techniques, that will be discussed later in this section. The difference is that early-pruning algorithms try to predict future candidate sequences, in an effort to avoid generating them in the first place.

2. **Search space partitioning:** This feature can be seen in some apriori-based algorithms like SPADE, in spite of being a characteristic of pattern-growth. As the name suggests it provides partitioning of the search space generated, being mostly used in large sets of candidate sequences. This feature plays an important role in pattern-growth algorithms, since it i s used by nearly everyone of them, with the goal to generate as few candidate sequences as possible. This in extent provides an efficient memory management. There are different ways to partition the search space. SPADE represents the search space as a *lattice* structure which will be discussed in more detail in later sections. Others such as WAP-mine[13] use a tree-projection structure (this algorithm will not be covered in this thesis).

3. **Suffix/Prefix growth:** Used in algorithms that are dependent on projected databases (PrefixSpan), it first starts by finding the frequent k-sequences and then chooses

each frequent item as a prefix or a suffix, after that the candidate sequences are built around these items and are mined recursively. This is done taking in account that frequent subsequences can always be found by growing a frequent prefix/suffix. This feature greatly reduces the amount of memory required to store the different candidate sequences that share the same prefix/suffix.

4. **Depth-first traversal:** Pattern-growth methods included into this construct each pattern separately, in a recursive way. All the subsequences are processed at each level before moving on to the next [17]. This feature uses less memory and less candidate sequence generation than breath-first search, having in extent a more directed search space. SPAM [5] is an algorithm that relies heavily in this feature. SPADE can also employ this method.

Early-Pruning: These types of algorithms are defined as using some sort of *position induction* to prune candidate sequences at a very early stage in mining. This technique used in the LAPIN algorithm as defined in [16], is used during the mining process and checks the items last position to see if it is smaller than the current prefix position. When this condition is verified the item cannot appear behind the current prefix in the same customer sequence. By doing this, the support counting is avoided as much as possible and the generation of infrequent candidate sequences is kept at a minimum. Features of this category are as follows:

1. **Support counting avoidance:** The idea behind this feature is to compute support of a candidate sequence without carrying a count throughout the mining process, and thus avoid scanning the sequence database each time it is needed to compute this support. The advantage of this feature is the removal of the sequence database from memory once it stores candidate sequences along with their corresponding support count.

2. **Vertical projection of the database:** This feature is also used in the SPADE algorithm [18] needing only to visit the sequence database once or twice, which then proceeds to construct a bitmap or position indication table for each frequent item, thus, obtaining a vertical layout of the database which is represented in tables 2.3 and 2.4. These tables are used in the mining process to generate candidate sequences and count their support. This feature has the advantage of reducing memory consumption, but on the other hand requires quite a lot computational power.

3. **Position-coded:** As refereed above, postion-coded (or position induction) is used in early-pruning algorithms to look-ahead and avoid generating infrequent candidate sequences, thus, avoiding the calculation of its support.

2.2.3 Structure definition

Being SPM a data-mining technique it has to follow the same structure as any data-mining algorithm would do. The operation cycle of data mining is divided in 3 stages [19]:

- **Preprocessing** - It is performed before any data-mining techniques are put into work, having the goal of cleaning and selecting the data for use from a raw database.

- **Data mining process** - At this stage the different data mining techniques are used in order to extract information and knowledge from the selected data.

- **Postprocessing** - Evaluates the mining result according to the user requirements.

The work-flow of the previously mentioned processes begins with cleaning and integrating the initial databases where we will get our raw data. The need to do so begin from the fact that information may come from several databases with different names to define the same concept, or even in the same database some inconsistencies may be found with missing or duplicate information. Suppose we have a supermarket that needs to check the sales revenue from a certain item in its store and there are lots of brands on sale but all of them have the same base vending item. In this case we need to group all sales of each brand together in order to have a notion of the big picture and get a general overview of the sales of the item to analyse.

Since not all of the data in the database is related to our mining task, after cleaning and integrating the data the next step is to select the task related data. Taking the example of this thesis we shape the information in order to obtain a sequential database, with sequences from telecommunication protocols, which will leave our database not only smaller, but ready to be mined.

The second process involves the use of data mining techniques applied to the selected data in order to extract different information or different knowledge [19]. The range of information extracted may vary depending on the adopted algorithm. In the particular case of Sequential pattern mining the knowledge we acquire consists in frequent item-set patterns.

Finally, we evaluate the knowledge achieved with the mining algorithm in order to assert that our goals have been reached. The evaluation may depend on what the end result is expected to have and varies with the user requirements.

2.2.4 Architectures

Being Sequential pattern mining a particular case of data mining, we can describe its architecture in a typical data mining system. Data mining can have a broad view in functionality. It is described in [8] as the process of discovering interesting knowledge from large amounts of data stored in databases, data warehouses, or other information repositories. So a typical data mining system can have several components distributed across the different processes associated to this technique. Taking into account the structure defined for SPM we can look at the architecture as follows:

- **Pre-processing:** This step intends to treat the data to be mined, so it starts with a *Data Repository*. This can be either one or a set of databases/other kind of information repositories where the information to be analysed is going to be extracted. Usually, the data contained on these repositories may be in a "raw" state, meaning that before extracting information, data cleaning and integration techniques may need to be done. These techniques involve removing any noise or inconsistent data that can be found in the initial repositories, combining distinct data sources into a whole and begin the selection of the pieces of data that are relevant for our mining task.
  After the initial "cleanse" the information extracted comes in the form of *clean data*. The data extracted is then transformed to a format that is ready to be mined, by performing different techniques such as summary or aggregation operations.

- **Data mining process:** This step is where the user interacts with the system, by giving the initial parameters to start the sequential pattern mining. These parameters can vary from minimal supports, to the selection of the Algorithm used.

- **Data Post-processing:** After applying the SPM techniques on the selected data, some knowledge will come out of that mining, in the form of frequent sequential patterns. These results are then evaluated, asserting if the outcome is the desired. If the information extracted is insufficient or has no substantial value to the problem the system is trying to solve, the data mining process repeats until these final conditions are reached. In the repeating process the user inserted parameters are changed in order to obtain different results.

Figure 2.1 shows a summarized visual representation of the core steps that are present in any sequential pattern mining architecture.

2.3 Sequential Pattern Mining Algorithms

This thesis will cover three of the most popular algorithms in sequential pattern mining, which will later be used to implement the protocol's behavioural analysis. The objective

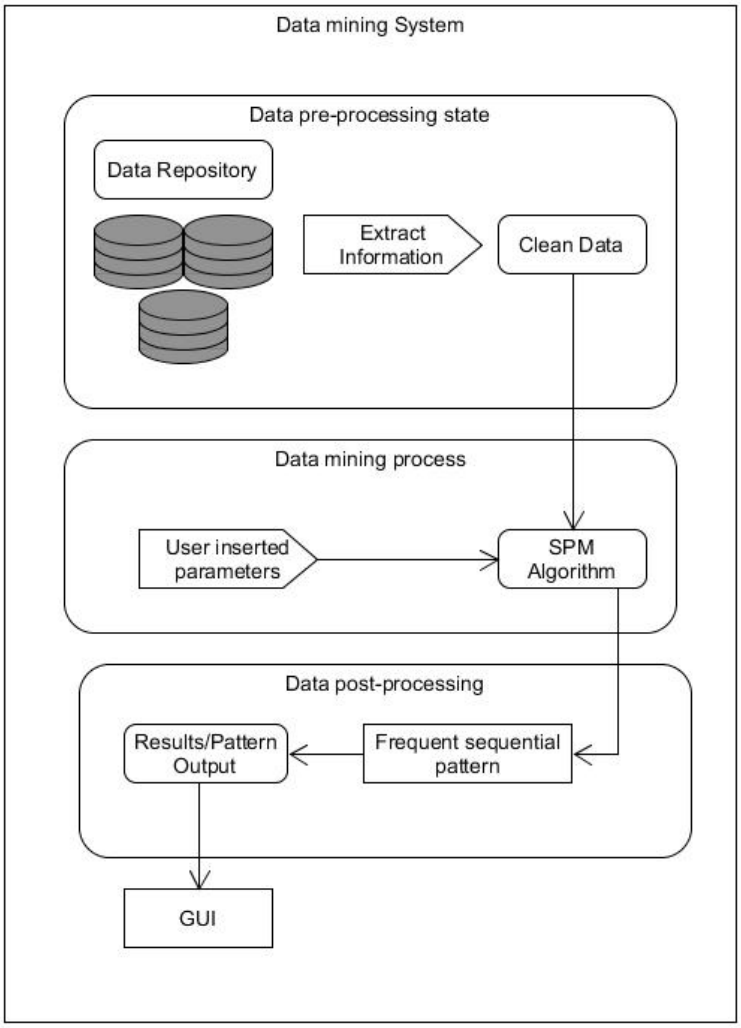

Figure 2.1: Sequential pattern mining Architecture

of this section is finding the differences between each of these implementations. It will include a brief overview of the selected algorithms, ending with several evaluation results capable of comparing their performance.

2.3.1 GSP

Generalized Sequential Patterns (GSP), proposed by Agrawal and Srikant (1996) [15], is categorized in the proposed taxonomy as an *apriori-based* algorithm. GSP uses the downward-closure property of sequential patterns and adopts a multiple-pass, candidate generate-and-test approach[11]. The first pass over the data determines the support of each item, which means it counts the number of occurrences of the item in the data-sequences. At the end of this first pass the algorithm knows which items pass the minimum support test, and consequently determine which of them are frequent. This pass generates a seed set which represents the frequent sequences found, that are to be used on

the next iteration of the algorithm. The seed set is used to generate candidate sequences that have one more item than the seed set, and consequently all candidate sequences will have the same number of items. The support of the candidate sequences is found during the pass over the data, being based on the minimum support. These candidate sequences are tested to determine if they are frequent or not. If they become frequent they will be used as the seed for the next candidate generation. The algorithm only stops when no frequent sequences nor candidate sequences are generated at the end of the pass.

GSP is described as having two sub-processes:

Sequence candidate generation: In this process candidate k-sequences are generated based on the set of all frequent (k-1)-sequences. First lets introduce the notion of a *contiguous subsequence* as it is presented in [15].

Definition Given a sequence $S = <S_1\ S_2 ... S_n>$ and a subsequence C, C is a *contiguous subsequence* of S if any of the following conditions hold:

1. C is derived from S by dropping an item from either S_1 or S_n.
2. C is derived from S by dropping an item from an element S_i which has at least 2 items.
3. C is a contiguous subsequence of C', and C' is a contiguous subsequence of S.

For example, consider the sequence S = <(ab)(cd)(e)(f)>. The sequences <(b)(cd)(e)>, <(ab)(c)(e)(f)> and <(c)(e)> are some of the contiguous sequences of S. However <(ab)(cd)(f)> and <(a)(e)(f)> are not.

Candidates are generated in two steps, similarly to the candidate generation procedure for finding association rules described in [2].

- **Join Phase**- here a sequence S_1 joins another sequence S_2 both belonging to the seed set L_k, if the subsequence obtained by dropping the first item in S_1 is the same as the subsequence formed by dropping the last items in S_2. The generated candidate sequence by joining S_1 and S_2 is the sequence S_1 extended by the last item of S_2 (e.g., *abcd* is the result of joining *abc* with *bcd*).

- **Prunning Phase**- those candidate sequences that have a contiguous subsequence whose support count is less than the minimal support (infrequent) are deleted.

An example of this can be seen in Table 2.2. In the join phase, the sequence <(1, 2)(3)> joins with <(2)(3, 4)> to generate <(1, 2)(3, 4)> and with <(2)(3)(5)> to generate <(1,2)(3)(5)>. In other hand the sequence <(1, 2)(4)> does not join with any sequence since there is no sequence of the form <(2)(4, x)>. In the prune phase the sequence <(1,2)(3)(5)> is dropped since there is no support for its contiguous subsequence <(1)(3)(5)>.

Table 2.2: Candidate Generation Example.

Frequent sequences	**After join**	**After prunning**
<(1, 2)(3)>	<(1, 2)(3, 4)>	<(1, 2)(3, 4)>
<(1, 2)(4)>	<(1, 2)(3)(5)>	
<(1)(3, 4)>		
<(1, 3)(5)>		
<(2)(3, 4)>		
<(2)(3)(5)>		

Candidate support count: This final sub-process starts by using an adaptation of the hash-tree structure of [2] to reduce the number of candidates to check the support count. After that the **Contain Test** is introduced to check if a data sequence D contained a candidate sequence S, being composed of two phases:

- **Forward phase:** GSP finds successive elements of the candidate sequence S in the data sequence D as long as the difference between the end-time of the element just found and the start-time of the previous element is less than the user defined time constrain *max-gap*. If the difference is more than max-gap, the algorithm switches to the backward phase. If an element is not found, the data-sequence does not contain S.

- **Backward phase:** The algorithm backtracks and "pulls up" previous elements. Suppose the current element is represent by S_i and the end-time $(S_i)=t$. The algorithm checks if there exists transactions containing S_{i-1} and their transaction-times are after t-*max-gap*. Since after pulling up S_{i-1} the difference between S_{i-1} and S_{i-2} may not satisfy the gap constraints, the algorithm moves backwards until either the max-gap constraint between the element just pulled up and the previous element is satisfied, or the first element has been pulled up. Then the algorithm switches to the forward phase. If any element can not be pulled up the data-sequence D does not contain S.

2.3.2 SPADE

SPADE is an algorithm developed by Zaki (2001) [18], and it relies on the *Lattice Theory* defined in [6]. The goal is to find frequent sequences using efficient lattice search techniques and simple join operations. All the existing sequences are usually discovered with only two or three passes over the database. Besides that, SPADE utilizes combinatorial properties to decompose the original problem into smaller portions, which can be

independently solved in main-memory. The sequential database is transformed into a vertical data format as in the example in Table 2.3, where the new database becomes a set of tuples of the form <*itemset* : (*sequence_ID*,*time_ID*)> [11] and each item id is associated with the corresponding sequence and time stamp. An example of the tuples present in the itemset <(a)> are (1,23), (6,6) and (21,2).

Table 2.3: Vertical database.

<(a)>	**<(b)>**	**<(c)>**	**<(d)>**	**<(e)>**
(SID , TID)	(SID , TID)	(SID , TID)	(SID , TID)	(SID , TID)
(1 , 23)	(2 , 36)	(1 , 26)	(6 , 12)	(6 , 23)
(6 , 6)	(7 , 18)			(21 , 26)
(21 , 2)				

SPADE can be seen has a three step method, first it scans the vertical database to compute frequent 1-sequences in an apriori-like way. For each database item the id-list is read from the disk to memory, and then the id-list is scanned and the support is incremented for each new sequence ID (SID) found.

The next step involves finding the frequent 2-sequences. A vertical-to-horizontal transformation is preformed on the database. For each item i, its id-list is scanned into memory. We then look for each (SID,TID) pair abbreviated as (s, t). We group those items with the TID (i,t) and order in increase of SID, and the resulting database is shown in table 2.4.

Table 2.4: Vertical to Horizontal database.

SID	**(Item,TID) pair**
1	(a,23) (d,26)
2	(a,19) (b,36)
6	(a,6) (e,12) (g,23)
7	(b,18)
21	(a,2) (h,26)

In the final step the 2-sequences are used to construct the lattice, the only problem is that the lattice can grow very big and cannot fit in the main memory. In order to solve this problem, Zaki (2001) [18] proposed that the lattice can be decomposed into disjoint subsets called *equivalence classes*, by assigning sequences with the same prefix items to the same class. With this partition each subset can be loaded and searched seperatly in main memory. The lattice is then traversed for support count and enumeration of frequent sequences[12]. There are two methods used to traverse the lattice, which are **breadth-first** (BF) and **depth-first** (DF) used in the SPAM algorithm [5].

In breadth-first search the classes are generated in a recursive bottom-up manner [19] (e.g., to generate the 3-item sequences all the 2-item sequences have to be processed). Depth-first search only needs one 2-item sequence and a k-item sequence to generate a $(k+1)$-item sequence. Pruning is also applied in this third step, using the apriori property [2] when intersecting id-lists (temporal joins) and counting support[12]. Comparing the two methods it can be concluded that Depth-first search dramatically reduces computation and the number of id-lists generated, needing less memory. However, breadth-first search has the advantage of having more information to prune candidate k-item sequences.

2.3.3 PrefixSpan

PrefixSpan [10] is based on the general idea of examining only the prefix subsequences and project only their corresponding postfix subsequences into projected databases. In each projected database, sequential patterns are grown by exploring only local frequent patterns. This algorithm employs database projection in order to shorten the database on the next pass, which in return speeds up computation. In this method there is no need for candidate generation, since it recursively projects the database according to their prefix.

To illustrate the idea of projected databases, consider <a>, <(ac)> and <(ac)h> from the sequential database in Table 2.5a. These are *prefixes* of the sequence <(ac)h(efg)>. On the other hand we can call <(_c)h(efg)> the *postfix* of the sequence <a>. An example of a Projected Database, assuming a support count of 2, can be seen in Table 2.5b.

Table 2.5: PrefixSpan database examples

Sequence ID	Sequence
1	<ab>
2	<(ac)h(efg)>
3	<(acb)>
4	<b(ci)a>
5	<dg>

(a) Sequence Database

Length-1 sequence	Projected Database
<a>	<b> ; <(_c)h(efg) ; <(_cb)>
<b>	<(ci)a>
<c>	<h(efg)> ; <b> ;<(_i)a>
<(ac)>	<d(efg)> ; <(_b)>
<g>	

(b) Level-by-Level Projected Database

There are three different projection methods that can be used in PrefixSpan, which are *level-by-level*, *bi-level* and finally *pseudo projection* [19]. The first method presented acts in three steps:

1. Scan the sequential database to obtain the length-1 sequences.

2. Divide the search space into different partitions, where each partition corresponds to the projection of the sequential database that contains the length-1 sequences as a prefix (e.g., for the frequent 1-sequence <a> as the prefix the projected database is shown in Table 2.5b). The projected database only contains the postifx of these sequences.

3. Find subsets of sequential patterns where these subsets can be mined by constructing projected databases, and mining each one recursively. This process is repeated until the projected database is empty or no more frequent length-k sequential patterns are generated.

In this method there is no candidate generation process, although it costs time and space to construct the projected databases.

The next method, *bi-level* projection, relies on the S-Matrix, introduced in [9], and was proposed to reduce the number and size of projected databases. It starts by performing the same first step as the previous method, which is finding all length-1 sequential patterns. Assuming the database S in Table 2.5 we have: <a>, <b>, <c>, <(ac)> and <g>. In the second step a 5x5 lower triangular matrix M is constructed as shown in Table 2.6, that represents all the supports for the length-2 sequences, for example M[a,c]=(2,1,2) counts the support for <ac>, <ca> and <(ac)> respectively. Since the information in cell M[c,a] is symmetric to that in M[a,c], a triangle matrix is sufficient, this matrix is called *S-Matrix*[10]. After this the S-Matrix for the frequent length-2 sequences is constructed, and the process are iterated until no other frequent sequences can be found or the projected database becomes empty. The use of this matrix provides fewer and smaller projected databases.

Table 2.6: S-Matrix

a	0				
b	(2,1,1)	0			
c	(2,1,2)	(1,1,1)	0		
(ac)	(0,0,0)	(0,1,1)	(0,0,0)	0	
g	(1,0,0)	(0,0,0)	(1,0,0)	(1,0,0)	0
	a	b	c	(ac)	g

Pseudo projection is used to make the projection more efficient when the projected database can be fitted in main memory. In this method no physical projected database is constructed in memory, instead each postfix (projection) is represented by two pieces of information: pointer to the sequence in the database and offset of the postfix in the sequence for each size of projection [12].

2.4 Conclusions

This section presents a comparison between the sequential pattern mining algorithms discussed before, focused on the computation time and memory consumption.

GSP is based on the *apriori-based* category, it explores a candidate generation-and-test approach [2] to reduce the number of candidates to be examined. However, this technique inherits some costs that are independent of the implementation techniques used [14].

1. **Excessive candidate sequence generation.** The set of candidate sequences includes all the possible permutation of elements and repetition of items in a sequence, which in return generates an explosive set of candidate sequences.

2. **Multiple scans of the database.** The length of a candidate sequence grows by one each time the database is scanned. In a short, to find the sequential pattern with a length of l the database needs to be scanned at least l times.

3. **Difficulty in mining long sequential patterns.** A long sequential pattern contains a combinatorial explosive number of subsequences, and such subsequences must be generated and tested in the a priori-based mining. Thus, the number of candidate sequences is exponential to the length of the sequential patterns to be mined.

The implications of these problems make *apriori-based* algorithms, such as GSP, computationally expensive due to maintaining the support count for each subsequence being mined, which leads to a huge memory consumption in the early stages of mining. The explosive number of candidate sequences also introduces I/O delay, due to the need to utilize a secondary storage in case the number of sequences gets to big. Despite that, GSP and its predecessor *AprioriAll* [1] generate all the sequential patterns although being inefficient. They are also the basis of further researches on mining sequential pattern, since they first introduced sequential pattern mining.

SPADE is classified as a *Apriori-Based and Pattern-Growth Hybrid* since it has key features from both classes. Experiments confirm that SPADE outperforms GSP by a factor of two [18]. It relies on the same generate-an-test feature from GSP, which in return leads to an excessive generation of candidate sequences. However, SPADE finds a solution to this problem by including some Pattern-Growth features such as search-space partitioning, which includes the vertical database layout defined in [17]. This technique reduces scans of the sequence database, making the algorithm faster. Also the inclusion of lattice structure partitioning allows to load the decomposed search space into main memory to be mined separately. This strategy provides a more efficient memory management and minimizes I/O costs [18].
On the other hand the basic search methodology of this algorithm relies on *breadth-first*

search and *Apriori pruning*. Despite the pruning, it has to generate large sets of candidates in breadth-first manner in order to grow longer sequences. Thus, most of the difficulties suffered in the GSP algorithm recur in SPADE as well [8]. One way to minimize this problem is to use a *depth-first search* strategy that requires less memory.

PrefixSpan falls under the *Pattern-Growth* category, and like many other algorithms of this type the key idea for this method is to avoid the candidate-generation step altogether, which is the most time consuming part of apriori algorithms such as GSP and AprioriAll [19]. It employs the database projection technique to make the database for the next pass much smaller turning the algorithm faster because the search space is reduced at each step (increasing the computation time in the presence of small support thresholds) [4]. This feature greatly reduces memory consumption in terms of candidate sequences.
The main cost of PrefixSpan lies in the projected database generation process, which can be computational expensive if the databases have to be generated physically. To solve this problem *pseudo-projection* was suggest to replace the physical projection of a sequence by a sequence identifier. However, this method can only be applied if the projected databases can be fitted into main memory. PrefixSpan also outperforms other algorithms if the sequential pattern is very long, since it can handle its treatment more efficiently.

A performance comparison of GSP, SPADE, and PrefixSpan shows that PrefixSpan has the best overall performance [14]. SPADE, although weaker than PrefixSpan in most cases, outperforms GSP. The comparison also found that when there is a large number of frequent subsequences the three algorithms run slowly [11].

Chapter 3

Work Plan

3.1 Task Description

The work plan of this thesis includes the following tasks:

- **Task 1:** Assembly of the prototype used to extract the data that will be mined by the selected sequential pattern mining algorithms. This thesis focus the SIP protocol. Therefore a SIP traffic generator will be used to simulate SIP signalling messages. The adopted traffic generator is **SIPp** that provides the possibility of creating custom scenarios. The generator will be operating in a Xubuntu environment and will communicate with an Asterisk server in order to exchange the messages between the simulated calls.

- **Task 2:** Definition of the custom scenarios that will be used to generate data. These scenarios will consist in simple SIP messages (e.g. INVITE, REGISTER), that will be modified in order to simulate errors in the system. The goal here is to obtain interesting data that could enrich our mining.

- **Task 3:** Extract the data generated. The traffic from the SIPp generator will be captured using Wireshark, and the logs will be processed extracting only the data that is useful for our mining techniques. This extraction will be done using the Ubuntu *grep* command in the console.

- **Task 4:** Adopt different data mining algorithms to extract the frequent patterns from the data retrieved by each of the used SIP scenarios. The goal is to use implementations of the algorithms and apply them to our data, whilst measuring the computational power and speed of each of the implementations.

- **Task 5:** Compare the performance of each of the algorithms in each of the scenarios. In this task a quantitative analysis will be made to check what kind of information and knowledge was produced, thus checking if any of the given patterns can give a better comprehension of the problems that are occurring.

3.2 Developed Work

This subsection will cover the goals that have been already reached in this thesis. The first task concluded was that of assembling the prototype for extracting data. The prototype consists in a SIP traffic generator (SIPp) that was installed in a Xubuntu machine, which connects to an access point. From here the traffic will be redirected to an asterisk server to process the SIP message and reply accordingly. The traffic generator was tested by running some integrated scenarios in the SIPp program which can simulate a User Agent Client and a User Agent Server.

The next step involved creating the custom scenarios that would be used to generate the data to be mined. The scenarios that were created involved a REGISTER, INVITE and a message digest leak that is present in the SIP protocol. These scenarios were created and modified to have errors, so our mining algorithms can detect patterns of error in each of the SIP methods.

3.3 Schedule

The schedule that was defined for this thesis is described as follows: performance

Figure 3.1: Schedule.

- **January:** Beginning of the SIPp installation to generate SIP traffic.
- **February:** Specify the scenarios that will be used in the thesis, being them prototype architecture and SIPp patterns.

- **March:** Beginning of the process of obtaining and cleaning data.
- **April:** Configure the algorithms. Here the goal is to have all the 3 algorithms ready to start mining our data. Once the algorithms are ready the data is ready to be mined.
- **May:** Finish the mining phase.
- **June:** Start analysing the knowledge extracted from the mining, checking its relevancy and performance of each of techniques.

Chapter 4

Conclusions

This thesis presents an implementation of sequential pattern mining in a telecommunications context, aiming to compare the performance between the different algorithms used in terms of speed and computational power. The goal here is to use sequential pattern mining techniques to find patterns in telecommunications signalling to indicate what type of behaviour at a give time. Some milestones have already been reached, including:

1. The study of 3 classical sequential pattern mining algorithms, describing each one in detail to better understand how they will behave when extracting information from the data gathered;

2. The implementation of a prototype to generate the data to be mined. Since the goal is to describe telecommunications protocol patterns, the selected protocol to be analysed for this thesis is SIP.

Having these problems already solved, the next step is to start generating enough data so our sequential pattern algorithms can begin mining properly. In this process the goal is to acquire relevant information that can accurately describe the behaviour of the given protocol. After this the implementation of the sequential pattern mining algorithms will be computed, which is aimed to be concluded by the beginning of May. In the end, near the beginning of June the analysis of the information mined will start, to evaluate the results of our mining task. This thesis aims to have proven the reliability of sequential pattern mining as a tool to detect telecommunications problems, and in result make their resolution easier.

Bibliography

[1] R. Agrawal and R. Srikant. "Mining sequential patterns." In: *Data Engineering, 1995. Proceedings of the Eleventh International Conference on*. IEEE. 1995.

[2] R. Agrawal, R. Srikant, et al. "Fast algorithms for mining association rules." In: *Proc. 20th int. conf. very large data bases, VLDB*. Vol. 1215. 1994.

[3] R. Agrawal, T. Imieliński, and A. Swami. "Mining association rules between sets of items in large databases." In: *Acm sigmod record*. Vol. 22. 2. ACM. 1993.

[4] C. Antunes and A. L. Oliveira. *Sequential pattern mining algorithms: trade-offs between speed and memory*. 2004.

[5] J. Ayres, J. Flannick, J. Gehrke, and T. Yiu. "Sequential pattern mining using a bitmap representation." In: *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM. 2002, pp. 429–435.

[6] B. A. Davey and H. A. Priestley. *Introduction to lattices and order*. Cambridge university press, 2002.

[7] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth. "From data mining to knowledge discovery in databases." In: *AI magazine* 17.3 (1996), p. 37.

[8] J. Han and M. Kamber. *Data Mining: Concepts and Techniques. Series Editor Morgan Kaufmann Publishers*. 2000.

[9] J. Han, J. Pei, B. Mortazavi-Asl, Q. Chen, U. Dayal, and M.-C. Hsu. "FreeSpan: frequent pattern-projected sequential pattern mining." In: *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM. 2000, pp. 355–359.

[10] J. Han, J. Pei, B. Mortazavi-Asl, H. Pinto, Q. Chen, U. Dayal, and M. Hsu. "Prefixspan: Mining sequential patterns efficiently by prefix-projected pattern growth." In: *proceedings of the 17th international conference on data engineering*. 2001.

[11] J. Han, H. Cheng, D. Xin, and X. Yan. "Frequent pattern mining: current status and future directions." In: *Data Mining and Knowledge Discovery* 15.1 (2007).

[12] N. R. Mabroukeh and C. I. Ezeife. "A taxonomy of sequential pattern mining algorithms." In: *ACM Computing Surveys (CSUR)* 43.1 (2010).

[13] J. Pei, J. Han, B. Mortazavi-Asl, and H. Zhu. "Mining access patterns efficiently from web logs." In: *Knowledge discovery and data mining. Current issues and new applications* (2000).

[14] J. Pei, J. Han, B. Mortazavi-Asl, J. Wang, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu. "Mining sequential patterns by pattern-growth: The prefixspan approach." In: *IEEE Transactions on knowledge and data engineering* 16.11 (2004), pp. 1424–1440.

[15] R. Srikant and R. Agrawal. "Mining sequential patterns: Generalizations and performance improvements." In: *Advances in Database Technology—EDBT'96* (1996).

[16] Z. Yang, Y. Wang, and M. Kitsuregawa. "LAPIN: Effective sequential pattern mining algorithms by last position induction." In: (2005).

[17] M. J. Zaki. "Efficient enumeration of frequent sequences." In: *Proceedings of the seventh international conference on Information and knowledge management*. ACM. 1998, pp. 68–75.

[18] M. J. Zaki. "SPADE: An efficient algorithm for mining frequent sequences." In: *Machine learning* 42.1 (2001).

[19] Q. Zhao and S. S. Bhowmick. "Sequential pattern mining: A survey." In: *ITechnical Report CAIS Nayang Technological University Singapore* (2003).

Ricardo Miguel Nunes Camacho de Matos

Degree in Electrical and Computer Engineering Sciences

Protocol's Behaviour Analysis

Dissertation plan submitted in partial fulfillment
of the requirements for the degree of

Master of Science in
Electrical and Computer Engineering

August, 2017

FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE NOVA DE LISBOA

Ricardo Miguel Nunes Camacho de Matos

Degree in Electrical and Computer Engineering Sciences

Protocol's Behaviour Analysis

Dissertation plan submitted in partial fulfillment
of the requirements for the degree of

Master of Science in
Electrical and Computer Engineering

August, 2017

Copyright © Ricardo Miguel Nunes Camacho de Matos, Faculty of Sciences and Technology, NOVA University Lisbon.
The Faculty of Sciences and Technology and the NOVA University Lisbon have the right, perpetual and without geographical boundaries, to file and publish this dissertation through printed copies reproduced on paper or on digital form, or by any other means known or that may be invented, and to disseminate through scientific repositories and admit its copying and distribution for non-commercial, educational or research purposes, as long as credit is given to the author and editor.

FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE NOVA DE LISBOA

Ricardo Matos
2017
Protocol's Behaviour Analysis
2017